Complete Presentation Script with Commands (Start to Finish)

Team Members: 1. Kushal G (PES1UG24AM145) | 2. Kshitij Satish Shetty (PES1UG24AM143)
Written in simple, clear English with the exact terminal commands inside every step.

Before Your Demo Starts (Run This 1 Command Before the Teacher Comes)

Open **Terminal #1** (PowerShell or CMD) inside `BigData_Fraud_Detection` and start the Simple Web UI:

```
python scripts/presentation_ui.py
```

*(This opens **http://localhost:8501** in your browser. Open a **second terminal window (Terminal #2)** inside `BigData_Fraud_Detection` to run the live commands below.)*

STEP 1 (0:00 – 0:45) | Introduction & Pipeline Flow — Kushal G (PES1UG24AM145)

What to Show on Screen: Top banner of the Simple Web UI (**http://localhost:8501**).

> "Good morning Sir/Ma'am. We are **Kushal (PES1UG24AM145)** and **Kshitij (PES1UG24AM143)**. Our project is a **Real-Time Credit Card Fraud Detection Pipeline**.
> When a customer swipes a credit card, the payment company has less than a second to decide whether it is a genuine purchase or fraud. Checking transactions in slow daily batches is too late because the money is already gone. So we built a 4-stage real-time streaming pipeline:
> 1. First, we use the **public ULB Credit Card Fraud dataset**, which contains **284,807 credit card swipes** and **492 actual frauds** (only 0.17% are fraud—about 1 fraud in every 580 swipes).
> 2. Second, our producer script replays the held-out test transactions as a live stream into **Apache Kafka**, which acts as our high-speed message pipe.
> 3. Third, **Apache Flink** reads the stream non-stop, checks that the event data is valid, and scores each transaction in real time using our trained **Logistic Regression fraud model**.
> 4. Fourth, Flink saves every scored transaction—along with its fraud probability and how many milliseconds scoring took—into an **Apache Iceberg** data warehouse table backed by MinIO storage."

STEP 2 (0:45 – 1:30) | Model Training, No-Cheating Proof & Code Tests — Kushal G (PES1UG24AM145)

Command to Run in Terminal #2:

```
python -m pytest -v
```

> "For our fraud model, we split the dataset **80% for training (227,845 rows)** and **20% for testing (56,962 rows)** using a fixed random seed (42) so our results are 100% reproducible. Because frauds are so rare, we used **StandardScaler** and **Logistic Regression with balanced class weights** so the model pays extra attention to fraud cases.
> We followed two strict rules:
> • First, we **strictly excluded the 'Class' fraud answer** from all model inputs so the model can never cheat. If any message tries to pass 'Class' inside the input features, our validation guard immediately rejects it.
> • Second, we exported the fitted scaling numbers and model weights into a JSON artifact (`models/fraud_model_artifact.json`) so Apache Flink calculates the exact same fraud probability natively. As you can see from our **14 automated tests passing in green**, Flink's math matches scikit-learn down to 16 decimal places (5.55×10^{-16})."

STEP 3 (1:30 – 2:15) | 3-Speed Streaming Load Test & Iceberg Storage — Kshitij (PES1UG24AM143)

Command to Run in Terminal #2 *(or click **'■ Run 3-Rate Test Now'** in Section 3 of the Web UI)*:

```
python scripts/smoke_test.py
```

"Next, we tested how our pipeline handles increasing traffic speeds. We replayed held-out transactions at three increasing rates: **200, 500, and 1,000 transactions per second**.
Every test run is tagged with its own **run_id**—and our Apache Iceberg table is partitioned by **run_id**—so transactions from different runs never mix. For every transaction, we subtract the producer send timestamp from the Flink scoring timestamp to measure the real latency in milliseconds (p50 median, p95, and p99).
We also injected **5 corrupted/malformed events** in each run (such as negative transaction amounts or missing features). Instead of crashing the pipeline, Flink catches those 5 bad events and isolates them into a separate **malformed_events** table, while saving **100% of the valid transactions** into Apache Iceberg with **0 lost and 0 duplicate records**."

STEP 4 (2:15 – 3:00) | Accuracy, Amount Segments & Who Pays the Cost — Kshitij (PES1UG24AM143)

Command to Run in Terminal #2 *(also show Section 1 & Section 2 in the Web UI at http://localhost:8501)*:

```
python scripts/run_evaluation.py
```

"Finally, we evaluated our model on all **56,962 held-out test transactions**, which contain **98 real frauds** and **56,864 normal swipes**.
Overall, at our default 0.50 threshold, the model catches **90 out of 98 frauds (91.84% Recall)** and misses only 8, with a **False-Positive Rate of just 2.44%** (1,386 false alarms) and a **PR-AUC of 0.7190**.
Because the public ULB dataset is anonymized for privacy and does not have customer names or ages, we grouped transactions into **3 proxy segments based on Transaction Amount** using training-data quantiles (**q33 = $9.99** and **q67 = $50.00**):
1. **Low Amount (≤ $9.99):** Highest **False-Positive Rate (3.40%, 664 false alarms)** and highest **False-Negative Rate (10.20%, 5 missed frauds)**. Half of all frauds (49 out of 98) happen here because hackers test stolen cards with tiny $1 swipes first. Here, **payment companies** pay the cost because reviewing a $2 false alarm costs more than the transaction fee, and **customers** get annoyed when small everyday purchases are blocked.
2. **Medium Amount ($9.99 to $50.00):** Lowest False-Positive Rate (**1.42%**) and highest **PR-AUC (0.8423)**.
3. **High Amount (> $50.00):** Best fraud detection—catching **94.74% of frauds** (only **5.26% False-Negative Rate**, missing just 2 out of 38 frauds). In high-amount purchases, **merchants and banks** bear huge financial chargeback losses if a fraud is missed (False Negative), while **merchants** also lose big sales if a real customer is falsely blocked (False Positive).
If a bank wants fewer false alarms, our threshold is configurable—raising it from 0.50 to 0.85 cuts false alarms by **72%** while still catching **89%** of all frauds. Thank you!"

Quick Summary of All Commands (In Order)

```
python -m pytest -v                  # 1. Run all 14 automated unit & verification tests
python scripts/smoke_test.py         # 2. Run 3-speed (200, 500, 1000 eps) Apache Iceberg test
python scripts/run_evaluation.py     # 3. Print accuracy (56,962 rows) & Low/Medium/High segments
python scripts/generate_report.py    # 4. Verify the 4-page technical report PDF
python scripts/presentation_ui.py    # 5. Launch Simple Web UI at http://localhost:8501
```

Common Teacher Viva Questions & Simple Answers

Teacher Question	Simple, Clear Answer to Give
1. Why use PR-AUC (0.719) instead of normal Accuracy?	Because 99.83% of swipes are normal! A dummy model that says 'normal' for everything gets 99.83% accuracy while catching 0 frauds. PR-AUC measures how well we catch the rare 0.17% frauds without raising too many false alarms.
2. Why is Recall 91.84% while Precision is 6.10%?	Because there are 578 normal swipes for every 1 fraud. Even a tiny 2.44% false-alarm rate on 56,864 normal swipes is 1,386 false alarms vs. 90 real frauds (90 / (90+1386) = 6.1%). Raising threshold to 0.85–0.95 boosts Precision to 39% and drops FPR to 0.24%.
3. Why group segments by Transaction Amount?	The ULB dataset hides personal customer details using PCA (V1–V28) for privacy, so there are no customer IDs or demographics. We used training-data Amount quantiles ($9.99 and $50.00) as reproducible transaction-amount proxies.
4. Who pays for False Positives vs. False Negatives?	False Positives (1,386): Cardholders get blocked at checkout, merchants lose sales, and payment companies pay review costs. False Negatives (8): Merchants and banks lose the stolen money and pay chargeback fees.
5. How do you stop different load-test runs from mixing?	Every event has a unique run_id (e.g. 200eps, 500eps, 1000eps), and the Apache Iceberg table is partitioned by run_id.